

LẬP TRÌNH CHO TRÍ TUỆ NHÂN TẠO

Giảng viên: Nguyễn Ngọc Giang

Pandas

1. Pandas là gì?

- Pandas: “Panel Data” hoặc “Python Data Analysis” , được tạo ra từ năm 2008 bởi Webs McKinney, là một thư viện Python mạnh mẽ hỗ trợ xử lý và phân tích dữ liệu.
- Pandas thường được sử dụng với dataset, giải quyết được 5 bước trong xử lý và phân tích dữ liệu:
 - Load
 - Prepare
 - Manipulate
 - Model
 - Analyze
- Được sử dụng rộng rãi trong khoa học dữ liệu, tài chính, kinh doanh và học máy.

1. Pandas là gì?

- Cung cấp hai cấu trúc dữ liệu chính: **Series** (mảng một chiều có nhãn) và **DataFrame** (bảng dữ liệu 2D).
- Pandas chủ yếu được viết bằng Python, còn lại những phần yêu cầu tính toán nhanh thì viết bằng Cython/C.
- Mã nguồn: <https://github.com/pandas-dev/pandas>

2. Cài đặt và khai báo

- Cài đặt Pandas: `pip install pandas`
- Import thư viện: `import pandas as pd`
- Kiểm tra phiên bản Pandas: `print(pd.__version__)`

Keyword to import a library

Keyword to refer to library by an alias (shortcut) name



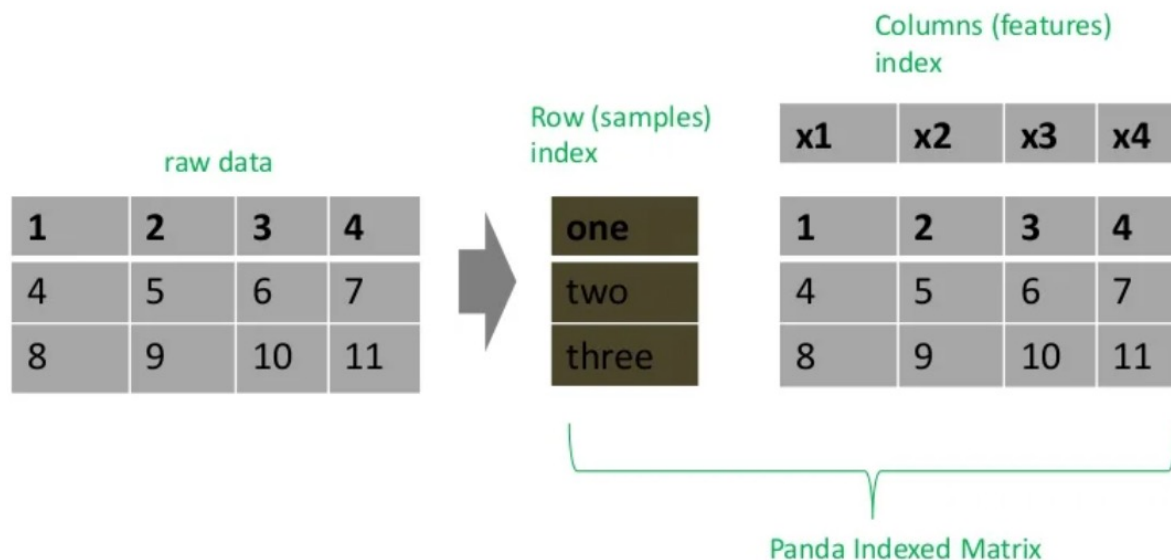
`import pandas as pd`

3. Kiểu dữ liệu trong pandas

Pandas Type	Native Python Type	Description
object	string	The most general dtype. Will be assigned to your column if column has mixed types (numbers and strings).
int64	int	Numeric characters. 64 refers to the memory allocated to hold this character.
float64	float	Numeric characters with decimals. If a column contains numbers and NaNs(see below), pandas will default to float64, in case your missing value has a decimal.
datetime64, timedelta[ns]	N/A (but see the datetime module in Python's standard library)	Values meant to hold time data. Look into these for time series experiments.

4. Thành phần chính trong pandas

Pandas are used to build **indexed arrays (1D)** and **matrices (2D)**, where columns and rows are labeled (named) and can be accessed via the labels (names).



- **2 thành phần chính là: Series và DataFrame**
 - Series: là mảng 1D được gán nhãn, có thể chứa các loại dữ liệu khác nhau
 - DataFrame: là bảng (ma trận) 2D được gán nhãn, có thể chứa các loại dữ liệu khác nhau

4.1 Cách dùng Series

- **Khởi tạo bởi nhiều cách khác nhau:**
 - Từ list, tuple, dictionary
 - Từ scalar value
 - Từ np.array
- **Cách truy cập phần tử:**
 - Dùng vị trí (position) như ndarray
 - Dùng nhãn (label/index)
 - Từ np.array

4.1. Cách dùng Series

Series Raw data Column Index Labels

↓ ↓ ↓

```
s = pd.Series( data, index=[ 'x1', 'x2', 'x3', 'x4' ] )
```

```
pandas.Series( data, index, dtype, copy)
```

Sr.No	Parameter & Description
1	data data takes various forms like ndarray, list, constants
2	index Index values must be unique and hashable, same length as data. Default np.arange(n) if no index is passed.
3	dtype dtype is for data type. If None, data type will be inferred
4	copy Copy data. Default False

4.1. Cách dùng Series

```
1 s1 = pd.Series([1, 2, 3.0])
2 print(s1)
```

```
0    1.0
1    2.0
2    3.0
dtype: float64
```

```
1 s2 = pd.Series([-2.3, "hello", 'banana', 2], index=[1, 2, 3, 4])
2 print(s2)
```

```
1    -2.3
2    hello
3    banana
4         2
dtype: object
```

```
1 s3 = pd.Series(0.3, index=["a", "b", "c", "d"])
2 print(s3)
```

```
a    0.3
b    0.3
c    0.3
d    0.3
dtype: float64
```

```
1 md = {
2     "key1": 0.3,
3     "key2": "wow",
4     "key3": -0.2
5 }
6 s4 = pd.Series(md)
7 print(s4)
```

```
key1    0.3
key2    wow
key3   -0.2
dtype: object
```

```
1 s = pd.Series((-0.3, 0.5, 1), index=["b", "a", "c", "d"])
2 print(s)
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-11-d437b27108a0> in <module>
----> 1 s = pd.Series((-0.3, 0.5, 1), index=["b", "a", "c", "d"])
      2 print(s)

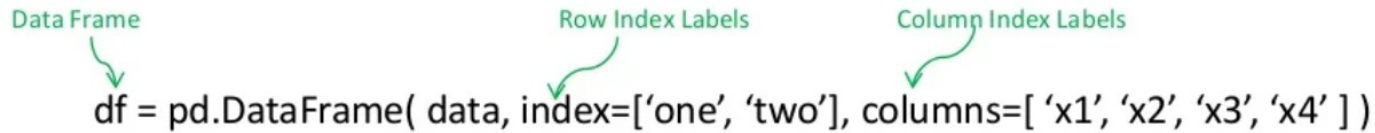
----- 1 frames -----
/usr/local/lib/python3.7/dist-packages/pandas/core/common.py in require_len
    530     if len(data) != len(index):
    531         raise ValueError(
--> 532             "Length of values "
    533             f"({len(data)}) "
    534             "does not match length of index "
```

ValueError: Length of values (3) does not match length of index (4)

4.2. Các dùng DataFrame

- **Khởi tạo bằng nhiều cách:**
 - Từ list, tuple, dict
 - Từ numpy array
 - Từ Series, DataFrame khác
 - Từ file (csv, excel, json,...)
 - Từ SQL database
- **Truy cập phần tử: Có nhiều cách**

4.2. Các dùng DataFrame


df = pd.DataFrame(data, index=['one', 'two'], columns=['x1', 'x2', 'x3', 'x4'])

```
pandas.DataFrame( data, index, columns, dtype, copy)
```

Sr.No	Parameter & Description
1	data data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame.
2	index For the row labels, the Index to be used for the resulting frame is Optional Default np.arange(n) if no index is passed.
3	columns For column labels, the optional default syntax is - np.arange(n). This is only true if no index is passed.
4	dtype Data type of each column.
5	copy This command (or whatever it is) is used for copying of data, if the default is False.

4.2. Các dùng DataFrame

```
1 df1 = pd.DataFrame()
2 print(df1)
```

```
Empty DataFrame
Columns: []
Index: []
```

```
1 data = ([ 'Alex',10], [ 'Bob',12], [ 'Clarke',13])
2 df3 = pd.DataFrame(data, columns=[ 'Name', 'Age'])
3 print(df3)
```

	Name	Age
0	Alex	10
1	Bob	12
2	Clarke	13

```
1 data = [101, 22, 333, 442, 554]
2 df2 = pd.DataFrame(data)
3 print(df2)
```

0	101
1	22
2	333
3	442
4	554

```
1 data = { 'Name': [ 'Tom', 'Jack', 'Steve', 'Ricky'],
2          'Age': [28,34,29,42]}
3 df5 = pd.DataFrame(data)
4 print(df5)
```

	Name	Age
0	Tom	28
1	Jack	34
2	Steve	29
3	Ricky	42

4.2. Các dùng DataFrame

```
1 data = {'Name': ['Tom', 'Jack', 'Steve', 'Ricky'],
2         'Age': [28, 34, 29, 42]}
3 df6 = pd.DataFrame(data, index=['rank1', 'rank2', 'rank3', 'rank4'])
4 print(df6)
```

	Name	Age
rank1	Tom	28
rank2	Jack	34
rank3	Steve	29
rank4	Ricky	42

```
1 data = [{'a': 1, 'b': 2},
2         {'a': 5, 'b': 10, 'c': 20}]
3 df7 = pd.DataFrame(data, index=["first", "second"])
4 print(df7)
```

	a	b	c
first	1	2	NaN
second	5	10	20.0

```
1 data = [{'a': 1, 'b': 2}, {'a': 5, 'b': 10, 'c': 20}]
2 |
3 df8 = pd.DataFrame(data, index=['first', 'second'], columns=['b', 'a'])
4 print(df8)
5
6 df9 = pd.DataFrame(data, index=['first', 'second'], columns=['a', 'c', 'b1'])
7 print(df9)
```

	b	a
first	2	1
second	10	5

	a	c	b1
first	1	NaN	NaN
second	5	20.0	NaN

```
1 data = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
2         'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}
3 df10 = pd.DataFrame(data)
4 print(df10)
```

	one	two
a	1.0	1
b	2.0	2
c	3.0	3
d	NaN	4

4.3 Thuộc tính của DataFrame object

df.attribute	description
dtypes	list the types of the columns
columns	list the column names
axes	list the row labels and column names
ndim	number of dimensions
size	number of elements
shape	return a tuple representing the dimensionality
values	numpy representation of the data

4.3 Thuộc tính của DataFrame object

```
In [4]: #Check a particular column type  
df['salary'].dtype
```

```
Out[4]: dtype('int64')
```

```
In [5]: #Check types for all the columns  
df.dtypes
```

```
Out[4]: rank          object  
discipline  object  
phd         int64  
service     int64  
sex         object  
salary     int64  
dtype: object
```

4.4 Các hàm của DataFrame object

- Có thể dùng hàm `dir()` để liệt kê toàn bộ thuộc tính và hàm trong DF: **`dir(df)`**

df.method()	description
head([n]), tail([n])	first/last n rows
describe()	generate descriptive statistics (for numeric columns only)
max(), min()	return max/min values for all numeric columns
mean(), median()	return mean/median values for all numeric columns
std()	standard deviation
sample([n])	returns a random sample of the data frame
dropna()	drop all the records with missing values

5. Đọc dữ liệu từ file

```
df = pd.read_csv('purchases.csv', index_col=0)
```

```
df = pd.read_json('purchases.json')
```

OUT:

	apples	oranges
June	3	0
Robert	2	3
Lily	0	7
David	1	2

5. Đọc dữ liệu từ file

- Hàm `read_excel()` trong pandas dùng thư viện `xlrd`

```
$ pip install xlrd
```

```
import pandas as pd
```

```
df = pd.read_excel('sample.xlsx')
```

```
Unnamed: 0   A   B   C
0      one  11  12  13
1      two  21  22  23
2     three  31  32  33
```

	A	B	C	D	E
1		A	B	C	
2	one	11	12	13	
3	two	21	22	23	
4	three	31	32	33	
5					
6					
7					
8					
9					
10					

```
df_sheet_multi = pd.read_excel('sample.xlsx', sheet_name=[0, 'sheet2'])
print(df_sheet_multi)
```

```
df_sheet_all = pd.read_excel('sample.xlsx', sheet_name=None)
print(df_sheet_all)
```

5. Đọc dữ liệu từ file

- Cần tạo kết nối dung thư viện thích hợp của python (pysqlite3).
- Truyền câu lệnh query vào pandas
- Ví dụ SQLite database có bảng tên là: purchases

```
pip install pysqlite3
```

```
!pip install pysqlite3
```

```
import sqlite3
```

```
con = sqlite3.connect("database.db")
```

```
df = pd.read_sql_query("SELECT * FROM purchases", con)
```

```
df = df.set_index('index')
```

OUT:

	index	apples	oranges
0	June	3	0
1	Robert	2	3
2	Lily	0	7
3	David	1	2

OUT:

	apples	oranges
index		
June	3	0
Robert	2	3
Lily	0	7
David	1	2

6. Ghi dữ liệu ra file

- Ghi DataFrame ra file CSV: `df.to_csv('output.csv', index=False)`
- Ghi DataFrame ra file Excel: `df.to_excel('output.xlsx', index=False)`

7. Xử lý dữ liệu với Pandas

- Lọc dữ liệu: `df[df['Age'] > 30]`

- Thêm cột mới:

```
df['Salary'] = [50000, 60000, 70000]
```

- Xóa cột: `df.drop(columns=['Salary'], inplace=True)`
- Kiểm tra dữ liệu bị thiếu: `df.isnull().sum()`

7. Xử lý dữ liệu với Pandas

○ Dataframe

loc vs iloc ?

- loc gets rows (and/or columns) with particular **labels**.
- iloc gets rows (and/or columns) at integer **locations**.

```
mac — IPython: Viz/Example_dataset — ipython — 58x25
In [15]: import pandas as pd
...:
...: data = {"Height": [1.8, 1.6, 1.5],
...:         "Weight": [50, 40, 45]}
...:
...: #load data into a DataFrame object:
...: df = pd.DataFrame(data, index = ['An', 'Tu', 'Tua
...: n'])
...: print(df)
      Height  Weight
An        1.8     50
Tu        1.6     40
Tuan       1.5     45

[In [16]: df.loc['Tu']
Out[16]:
Height      1.6
Weight     40.0
Name: Tu, dtype: float64

[In [17]: df.iloc[1]
Out[17]:
Height      1.6
Weight     40.0
Name: Tu, dtype: float64
```

7. Xử lý dữ liệu với Pandas

○ Dataframe

Common attributes

head, tail, info, columns

```
mac — IPython: Viz/Example_dataset — ipython — 58x25

[In [27]: df.info()
<class 'pandas.core.frame.DataFrame'>
Index: 3 entries, An to Tuan
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   Height  3 non-null      float64
1   Weight  3 non-null      int64
dtypes: float64(1), int64(1)
memory usage: 180.0+ bytes

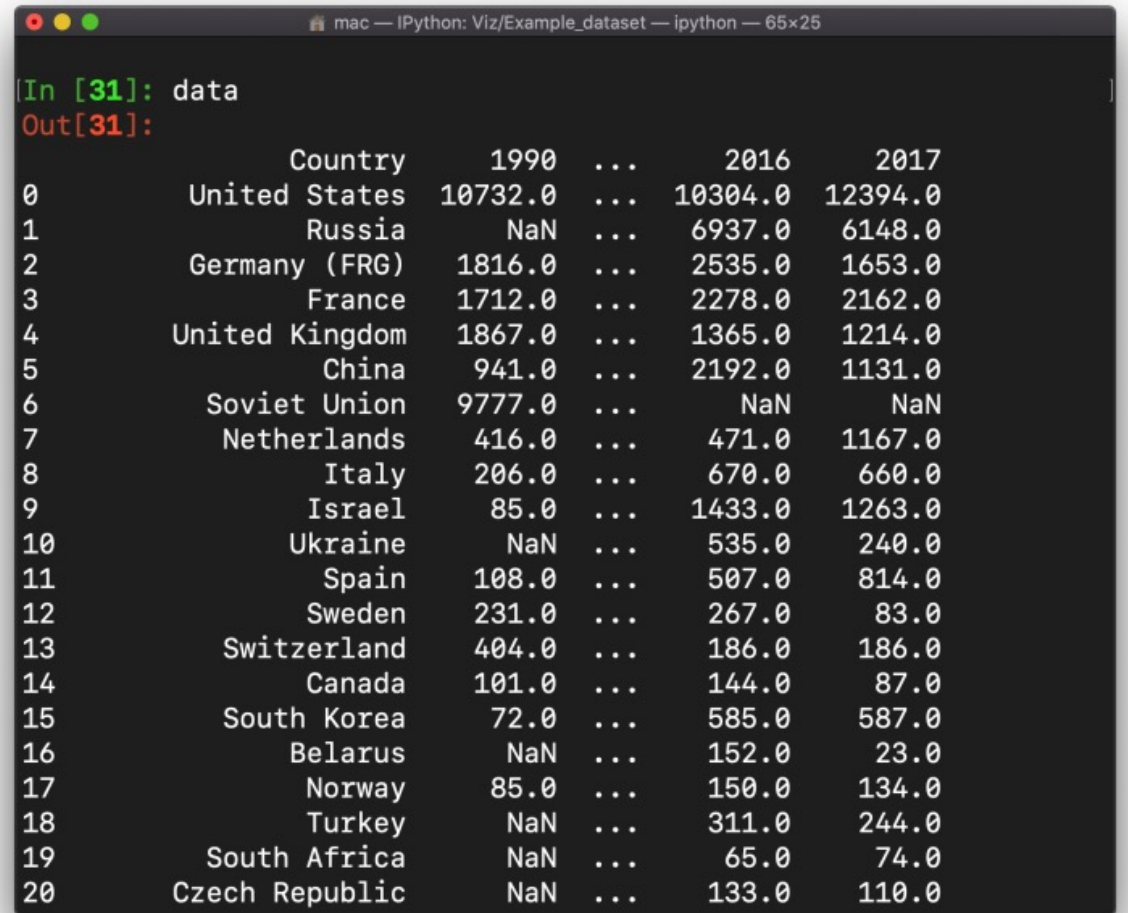
[In [28]: df.columns
Out[28]: Index(['Height', 'Weight'], dtype='object')

In [29]:
```

8. Data cleaning với Pandas

○ Data needs to be cleaned

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates
- Outline data



The screenshot shows a Jupyter Notebook terminal window with the following content:

```
mac — IPython: Viz/Example_dataset — ipython — 65x25
[In [31]: data
Out[31]:
```

	Country	1990	...	2016	2017
0	United States	10732.0	...	10304.0	12394.0
1	Russia	NaN	...	6937.0	6148.0
2	Germany (FRG)	1816.0	...	2535.0	1653.0
3	France	1712.0	...	2278.0	2162.0
4	United Kingdom	1867.0	...	1365.0	1214.0
5	China	941.0	...	2192.0	1131.0
6	Soviet Union	9777.0	...	NaN	NaN
7	Netherlands	416.0	...	471.0	1167.0
8	Italy	206.0	...	670.0	660.0
9	Israel	85.0	...	1433.0	1263.0
10	Ukraine	NaN	...	535.0	240.0
11	Spain	108.0	...	507.0	814.0
12	Sweden	231.0	...	267.0	83.0
13	Switzerland	404.0	...	186.0	186.0
14	Canada	101.0	...	144.0	87.0
15	South Korea	72.0	...	585.0	587.0
16	Belarus	NaN	...	152.0	23.0
17	Norway	85.0	...	150.0	134.0
18	Turkey	NaN	...	311.0	244.0
19	South Africa	NaN	...	65.0	74.0
20	Czech Republic	NaN	...	133.0	110.0

8. Data cleaning với Pandas

○ Common attributes

<code>df.dropna()</code>	remove rows that contain empty cells
<code>df.fillna()</code>	replace empty cells with a value
<code>df.sort_value()</code>	sort data by specific column
<code>df.groupby()</code>	Splitting the data into groups/Applying a function to each group/ Combining the results into a data structure.
<code>df.join()/merge()/concat()</code>	combining together Series or DataFrame
<code>df.rename()</code>	Rename Specific Columns

8. Data cleaning với Pandas

○ `df.rename()`

```
mac — IPython: Viz/Example_dataset — ipython — 57x30

In [84]: import pandas as pd
...:
...: data = {"Height": [1.8, 1.6, 1.5],
...:         "Weight": [50, 40, 45]}
...:
...: #load data into a DataFrame object:
...: df = pd.DataFrame(data)
...:
...: print(df)
   Height  Weight
0     1.8     50
1     1.6     40
2     1.5     45

In [85]: df.rename(columns={'Height': 'Chieu_Cao'}, inplace=True)

In [86]: df
Out[86]:
   Chieu_Cao  Weight
0         1.8     50
1         1.6     40
2         1.5     45

In [87]:
```

8. Data cleaning với Pandas

○ df.dropna()

```
[In [52]: data = pd.read_csv('sub_data.csv')]
```

```
[In [53]: data]
```

```
Out[53]:
```

	Country	1990	1991	1992
0	United States	10732.0	12515.0	14132.0
1	Russia	NaN	NaN	2605.0
2	Germany (FRG)	1816.0	2482.0	1437.0
3	France	1712.0	1111.0	1185.0
4	United Kingdom	1867.0	1523.0	1180.0
5	China	941.0	1315.0	703.0
6	Soviet Union	9777.0	5667.0	NaN
7	Netherlands	416.0	455.0	365.0
8	Italy	206.0	349.0	248.0
9	Israel	85.0	152.0	324.0
10	Ukraine	NaN	NaN	165.0
11	Spain	108.0	100.0	65.0
12	Sweden	231.0	156.0	149.0
13	Switzerland	404.0	432.0	396.0
14	Canada	101.0	121.0	143.0
15	South Korea	72.0	118.0	NaN
16	Belarus	NaN	NaN	NaN
17	Norway	85.0	170.0	75.0
18	Turkey	NaN	NaN	NaN
19	South Africa	NaN	54.0	75.0

```
In [54]:
```

```
[In [56]: data.dropna(inplace=True)]
```

```
[In [57]: data]
```

```
Out[57]:
```

	Country	1990	1991	1992
0	United States	10732.0	12515.0	14132.0
2	Germany (FRG)	1816.0	2482.0	1437.0
3	France	1712.0	1111.0	1185.0
4	United Kingdom	1867.0	1523.0	1180.0
5	China	941.0	1315.0	703.0
7	Netherlands	416.0	455.0	365.0
8	Italy	206.0	349.0	248.0
9	Israel	85.0	152.0	324.0
11	Spain	108.0	100.0	65.0
12	Sweden	231.0	156.0	149.0
13	Switzerland	404.0	432.0	396.0
14	Canada	101.0	121.0	143.0
17	Norway	85.0	170.0	75.0

```
In [58]:
```

8. Data cleaning với Pandas

○ df.fillna()

```
mac — IPython: Viz/Example_dataset — ipython — 57x28

In [52]: data = pd.read_csv('sub_data.csv')

In [53]: data
Out[53]:
```

	Country	1990	1991	1992
0	United States	10732.0	12515.0	14132.0
1	Russia	NaN	NaN	2605.0
2	Germany (FRG)	1816.0	2482.0	1437.0
3	France	1712.0	1111.0	1185.0
4	United Kingdom	1867.0	1523.0	1180.0
5	China	941.0	1315.0	703.0
6	Soviet Union	9777.0	5667.0	NaN
7	Netherlands	416.0	455.0	365.0
8	Italy	206.0	349.0	248.0
9	Israel	85.0	152.0	324.0
10	Ukraine	NaN	NaN	165.0
11	Spain	108.0	100.0	65.0
12	Sweden	231.0	156.0	149.0
13	Switzerland	404.0	432.0	396.0
14	Canada	101.0	121.0	143.0
15	South Korea	72.0	118.0	NaN
16	Belarus	NaN	NaN	NaN
17	Norway	85.0	170.0	75.0
18	Turkey	NaN	NaN	NaN
19	South Africa	NaN	54.0	75.0

```
In [54]:
```

```
mac — IPython: Viz/Example_dataset — ipython — 57x28

In [59]: data = pd.read_csv('sub_data.csv')

In [60]: data.fillna(0, inplace=True)

In [61]: data
Out[61]:
```

	Country	1990	1991	1992
0	United States	10732.0	12515.0	14132.0
1	Russia	0.0	0.0	2605.0
2	Germany (FRG)	1816.0	2482.0	1437.0
3	France	1712.0	1111.0	1185.0
4	United Kingdom	1867.0	1523.0	1180.0
5	China	941.0	1315.0	703.0
6	Soviet Union	9777.0	5667.0	0.0
7	Netherlands	416.0	455.0	365.0
8	Italy	206.0	349.0	248.0
9	Israel	85.0	152.0	324.0
10	Ukraine	0.0	0.0	165.0
11	Spain	108.0	100.0	65.0
12	Sweden	231.0	156.0	149.0
13	Switzerland	404.0	432.0	396.0
14	Canada	101.0	121.0	143.0
15	South Korea	72.0	118.0	0.0
16	Belarus	0.0	0.0	0.0
17	Norway	85.0	170.0	75.0
18	Turkey	0.0	0.0	0.0
19	South Africa	0.0	54.0	75.0

8. Data cleaning với Pandas

○ `df.groupby()`

```
mac — IPython: Viz/Example_dataset — ipython — 57x30
In [72]: data = pd.read_csv('10_OneNumSevCatSubgroupsSev0
...: bs.csv')

In [73]: data
Out[73]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
..	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

```
[244 rows x 7 columns]

In [74]: grouped = data.groupby('sex')

In [75]: grouped.count()
Out[75]:
```

	total_bill	tip	smoker	day	time	size
sex						
Female	87	87	87	87	87	87
Male	157	157	157	157	157	157

```
In [76]:
```

8. Data cleaning với Pandas

Merge()

- `pd.merge(data1, data2, how='left', on='X1')`

data1

X1	X2
a	11.432
b	1.303
c	99.906

data2

X1	X3
a	20.784
b	NaN
d	20.784

X1	X2	X3
a	11.432	20.784
b	1.303	NaN
c	99.906	NaN

8. Data cleaning với Pandas

Merge()

- `pd.merge(data1, data2, how='right', on='X1')`

data1

X1	X2
a	11.432
b	1.303
c	99.906

data2

X1	X3
a	20.784
b	NaN
d	20.784

X1	X2	X3
a	11.432	20.784
b	1.303	NaN
d	NaN	20.784

8. Data cleaning với Pandas

Merge()

- `pd.merge(data1, data2, how='inner', on='X1')`

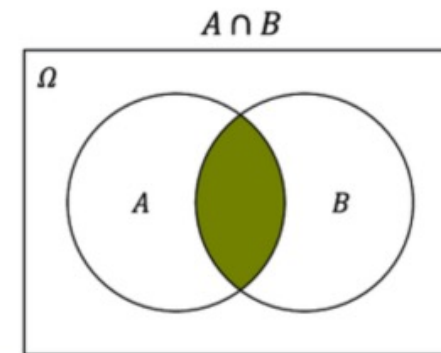
data1

X1	X2
a	11.432
b	1.303
c	99.906

data2

X1	X3
a	20.784
b	NaN
d	20.784

X1	X2	X3
a	11.432	20.784
b	1.303	NaN



8. Data cleaning với Pandas

Merge()

- `pd.merge(data1, data2, how='outer', on='X1')`

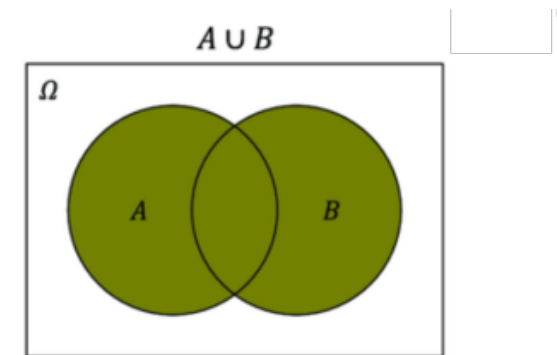
data1

X1	X2
a	11.432
b	1.303
c	99.906

data2

X1	X3
a	20.784
b	NaN
d	20.784

X1	X2	X3
a	11.432	20.784
b	1.303	NaN
c	99.906	NaN
d	NaN	20.784



8. Data cleaning với Pandas

Join()

- `data1.join(data2, how='right')`

Concatenate()

- Vertical
 - ➔ `s.append(s2)`
- Horizontal
 - ➔ `pd.concat([s,s2],axis=1, keys=['One','Two'])`
 - ➔ `pd.concat([data1, data2], axis=1, join='inner')`

8. Data cleaning với Pandas

Pivot() / Pivot_table()

- `df3= df2.pivot(index='Date', columns='Type', values='Value')`

- Spread rows into columns

	Date	Type	Value
0	2016-03-01	a	11.432
1	2016-03-02	b	13.031
2	2016-03-01	c	20.784
3	2016-03-03	a	99.906
4	2016-03-02	a	1.303
5	2016-03-03	c	20.784



Type	a	b	c
Date			
2016-03-01	11.432	NaN	20.784
2016-03-02	1.303	13.031	NaN
2016-03-03	99.906	NaN	20.784

8. Data cleaning với Pandas

Stack() / Unstack()

- `stacked = df5.stack()`
- `stacked.unstack()`
- Pivot a level of column labels
- Pivot a level of index labels

		0	1
1	5	0.233482	0.390959
2	4	0.184713	0.237102
3	3	0.433522	0.429401

Unstacked



1	5	0	0.233482
		1	0.390959
2	4	0	0.184713
		1	0.237102
3	3	0	0.433522
		1	0.429401

Stacked

8. Data cleaning với Pandas

Melt()

- `pd.melt(df2, id_vars=["Date"], value_vars=["Type", "Value"], value_name="Observations")`
- Gather columns into rows

	Date	Type	Value
0	2016-03-01	a	11.432
1	2016-03-02	b	13.031
2	2016-03-01	c	20.784
3	2016-03-03	a	99.906
4	2016-03-02	a	1.303
5	2016-03-03	c	20.784



	Date	Variable	Observations
0	2016-03-01	Type	a
1	2016-03-02	Type	b
2	2016-03-01	Type	c
3	2016-03-03	Type	a
4	2016-03-02	Type	a
5	2016-03-03	Type	c
6	2016-03-01	Value	11.432
7	2016-03-02	Value	13.031
8	2016-03-01	Value	20.784
9	2016-03-03	Value	99.906
10	2016-03-02	Value	1.303
11	2016-03-03	Value	20.784

9. Tổng kết

- Pandas giúp xử lý dữ liệu một cách nhanh chóng và hiệu quả.
- Series và DataFrame là hai cấu trúc dữ liệu chính.
- Có thể dễ dàng đọc, ghi và thao tác dữ liệu với Pandas.

Data Wrangling

with pandas Cheat Sheet
<http://pandas.pydata.org>

Pandas [API Reference](#) Pandas [User Guide](#)

Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

	a	b	c
N	1	4	7
D	2	5	8
E	2	6	9

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = pd.MultiIndex.from_tuples(
        [('d', 1), ('d', 2),
         ('e', 2)], names=['n', 'v']))
```

Create DataFrame with a MultiIndex

Method Chaining

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
      .rename(columns={
          'variable': 'var',
          'value': 'val'})
      .query('val >= 200'))
```

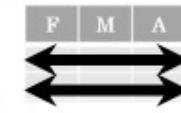
Tidy Data – A foundation for wrangling in pandas

In a tidy data set:



Each **variable** is saved in its own **column**

&



Each **observation** is saved in its own **row**

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.



M * A

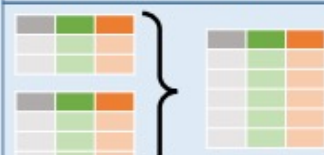
Reshaping Data – Change layout, sorting, reindexing, renaming



`pd.melt(df)`
Gather columns into rows.



`df.pivot(columns='var', values='val')`
Spread rows into columns.



`pd.concat([df1, df2])`
Append rows of DataFrames



`pd.concat([df1, df2], axis=1)`
Append columns of DataFrames

`df.sort_values('mpg')`
Order rows by values of a column (low to high).

`df.sort_values('mpg', ascending=False)`
Order rows by values of a column (high to low).

`df.rename(columns = {'y': 'year'})`
Rename the columns of a DataFrame

`df.sort_index()`
Sort the index of a DataFrame

`df.reset_index()`
Reset index of DataFrame to row numbers, moving index to columns.

`df.drop(columns=['Length', 'Height'])`
Drop columns from DataFrame

Subset Observations - rows



`df[df.Length > 7]`
Extract rows that meet logical criteria.

`df.drop_duplicates()`
Remove duplicate rows (only considers columns).

`df.sample(frac=0.5)`
Randomly select fraction of rows.

`df.sample(n=10)` Randomly select n rows.

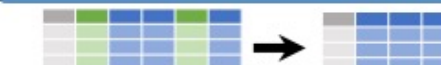
`df.nlargest(n, 'value')`
Select and order top n entries.

`df.nsmallest(n, 'value')`
Select and order bottom n entries.

`df.head(n)`
Select first n rows.

`df.tail(n)`
Select last n rows.

Subset Variables - columns



`df[['width', 'length', 'species']]`
Select multiple columns with specific names.

`df['width']` or `df.width`
Select single column with specific name.

`df.filter(regex='regex')`
Select columns whose name matches regular expression `regex`.

Using query

`query()` allows Boolean expressions for filtering rows.

`df.query('Length > 7')`

`df.query('Length > 7 and Width < 8')`

`df.query('Name.str.startswith("abc")', engine="python")`

Subsets - rows and columns

Use `df.loc[]` and `df.iloc[]` to select only rows, only columns or both.

Use `df.at[]` and `df.iat[]` to access a single value by row and column.

First index selects rows, second index columns.

`df.iloc[10:20]`
Select rows 10-20.

`df.iloc[:, [1, 2, 5]]`
Select columns in positions 1, 2 and 5 (first column is 0).

`df.loc[:, 'x2': 'x4']`
Select all columns between x2 and x4 (inclusive).

`df.loc[df['a'] > 10, ['a', 'c']]`
Select rows meeting logical condition, and only the specific columns.

`df.iat[1, 2]` Access single value by index

`df.at[4, 'A']` Access single value by label

Logic in Python (and pandas)

<	Less than	<code>!=</code>	Not equal to
>	Greater than	<code>df.column.isin(values)</code>	Group membership
==	Equals	<code>pd.isnull(obj)</code>	Is NaN
<=	Less than or equals	<code>pd.notnull(obj)</code>	Is not NaN
>=	Greater than or equals	<code>&, , ~, ^, df.any(), df.all()</code>	Logical and, or, not, xor, any, all

regex (Regular Expressions) Examples

<code>'\.'</code>	Matches strings containing a period '.'
<code>'Length\$'</code>	Matches strings ending with word 'Length'
<code>'^Sepal'</code>	Matches strings beginning with the word 'Sepal'
<code>'^x[1-5]\$'</code>	Matches strings beginning with 'x' and ending with 1,2,3,4,5
<code>'^(?!Species\$).*\$'</code>	Matches strings except the string 'Species'

Summarize Data

df['w'].value_counts()

Count number of rows with each unique value of variable

len(df)

of rows in DataFrame.

df.shape

Tuple of # of rows, # of columns in DataFrame.

df['w'].nunique()

of distinct values in a column.

df.describe()

Basic descriptive and statistics for each column (or GroupBy).



pandas provides a large set of **summary functions** that operate on different kinds of pandas objects (DataFrame columns, Series, GroupBy, Expanding and Rolling (see below)) and produce single values for each of the groups. When applied to a DataFrame, the result is returned as a pandas Series for each column. Examples:

sum()

Sum values of each object.

count()

Count non-NA/null values of each object.

median()

Median value of each object.

quantile([0.25,0.75])

Quantiles of each object.

apply(function)

Apply function to each object.

min()

Minimum value in each object.

max()

Maximum value in each object.

mean()

Mean value of each object.

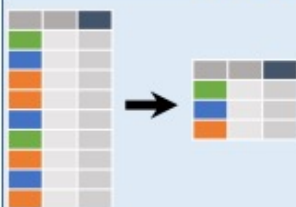
var()

Variance of each object.

std()

Standard deviation of each object.

Group Data



df.groupby(by="col")

Return a GroupBy object, grouped by values in column named "col".

df.groupby(level="ind")

Return a GroupBy object, grouped by values in index level named "ind".

All of the summary functions listed above can be applied to a group. Additional GroupBy functions:

size()

Size of each group.

agg(function)

Aggregate group using function.

Windows

df.expanding()

Return an Expanding object allowing summary functions to be applied cumulatively.

df.rolling(n)

Return a Rolling object allowing summary functions to be applied to windows of length n.

Handling Missing Data

df.dropna()

Drop rows with any column having NA/null data.

df.fillna(value)

Replace all NA/null data with value.

Make New Columns



df.assign(Area=lambda df: df.Length*df.Height)

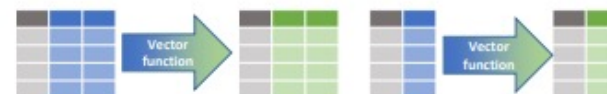
Compute and append one or more new columns.

df['Volume'] = df.Length*df.Height*df.Depth

Add single column.

pd.qcut(df.col, n, labels=False)

Bin column into n buckets.



pandas provides a large set of **vector functions** that operate on all columns of a DataFrame or a single selected column (a pandas Series). These functions produce vectors of values for each of the columns, or a single Series for the individual Series. Examples:

max(axis=1)

Element-wise max.

clip(lower=-10, upper=10)

Trim values at input thresholds

min(axis=1)

Element-wise min.

abs()

Absolute value.

The examples below can also be applied to groups. In this case, the function is applied on a per-group basis, and the returned vectors are of the length of the original DataFrame.

shift(1)

Copy with values shifted by 1.

rank(method='dense')

Ranks with no gaps.

rank(method='min')

Ranks. Ties get min rank.

rank(pct=True)

Ranks rescaled to interval [0, 1].

rank(method='first')

Ranks. Ties go to first value.

shift(-1)

Copy with values lagged by 1.

cumsum()

Cumulative sum.

cummax()

Cumulative max.

cummin()

Cumulative min.

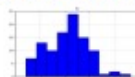
cumprod()

Cumulative product.

Plotting

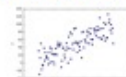
df.plot.hist()

Histogram for each column



df.plot.scatter(x='w', y='h')

Scatter chart using pairs of points



Combine Data Sets



Standard Joins

pd.merge(adf, bdf, how='left', on='x1')
Join matching rows from bdf to adf.

pd.merge(adf, bdf, how='right', on='x1')
Join matching rows from adf to bdf.

pd.merge(adf, bdf, how='inner', on='x1')
Join data. Retain only rows in both sets.

pd.merge(adf, bdf, how='outer', on='x1')
Join data. Retain all values, all rows.

Filtering Joins

adf[adf.x1.isin(bdf.x1)]
All rows in adf that have a match in bdf.

adf[~adf.x1.isin(bdf.x1)]
All rows in adf that do not have a match in bdf.



Set-like Operations

pd.merge(ydf, zdf)
Rows that appear in both ydf and zdf (Intersection).

pd.merge(ydf, zdf, how='outer')
Rows that appear in either or both ydf and zdf (Union).

pd.merge(ydf, zdf, how='outer', indicator=True)
.query('_merge == "left_only"')
.drop(columns=['_merge'])
Rows that appear in ydf but not zdf (Setdiff).

10. Link thực hành colab trên lớp

- [https://colab.research.google.com/drive/1-sORKb41lrIOpQVFyPR9MG9KacZFF6q ?usp=sharing](https://colab.research.google.com/drive/1-sORKb41lrIOpQVFyPR9MG9KacZFF6q?usp=sharing)

11. Bài tập về nhà

- Chọn bài tập trong link github sau, mỗi folder chọn lấy 1 bài tập (Từ folder 00 đến folder 09)
- <https://github.com/thieu1995/pandas>